Cómo trabajo con IA

Caso técnico Cobre · Notificator · Martin Rojo · octubre 2026

Este documento resume cómo usé IA para construir el proyecto: qué herramientas, con qué flujo, cómo escribo los prompts y qué aprendí en el camino. Los ejemplos son prompts reales de las sesiones de trabajo: capturas de Claude Code o texto copiado tal cual.

Idea central: la IA escribe código y artefactos, pero las decisiones de diseño son mías. Para que eso funcione, las decisiones tienen que quedar escritas en un lugar que la IA lea en cada sesión, y cada cambio tiene que pasar por una etapa de discusión antes de tocar código.

1. Herramientas

Herramienta	Para qué la uso
Claude Code (extensión de VS Code)	Agente que lee el repo, ejecuta comandos (mvn, docker compose, curl) y edita código.
OpenSpec	Desarrollo guiado por specs: cada cambio es una carpeta con proposal, design, specs y tasks que se revisa antes de implementar.
Comandos `/opsx:*`	Skills de OpenSpec instaladas en `.claude/`: explore, propose, apply, archive, sync, update.
`openspec/config.yaml`	La memoria del proyecto: dominio, stack, convenciones y decisiones que valen para todos los cambios.
Plugin Ponytail	Pide a la IA la solución más simple que funcione y evita abstracciones especulativas.

2. El flujo con OpenSpec

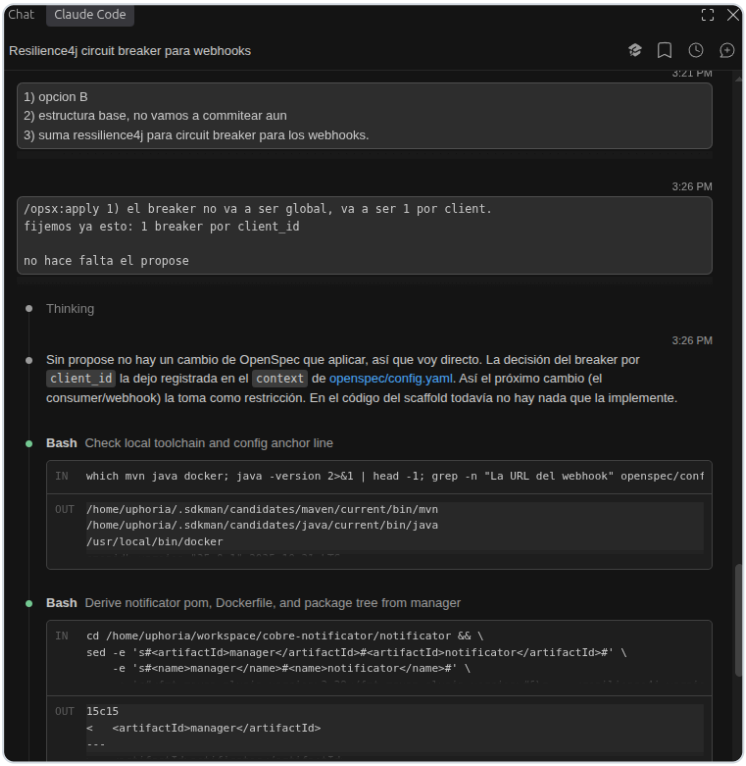
explore → **propose** → **apply** → **archive**

Paso	Qué pasa	Qué hago yo
explore	La IA lee el código y discute alternativas. No escribe código.	Planteo el problema, pido trade-offs y cuestiono lo que no me cierra. Acá se toman las decisiones.
propose	Genera `proposal.md`, `design.md`, specs con escenarios GIVEN/WHEN/THEN y `tasks.md`.	Reviso los artefactos y corrijo antes de implementar. Es más barato corregir un design que un PR.
apply	Implementa las tareas una por una, con tests, y marca los checkboxes.	Verifico contra el sistema real: compose levantado, curl, métricas y dashboards.
archive	Mueve el change a `archive/` y sincroniza las specs principales.	Las specs en `openspec/specs/` quedan como documentación viva del comportamiento.

Changes archivados hasta ahora: `manager-skeleton`, `list-notification-events`, `get-notification-event`, `add-kafka`, `add-manager-replay`, `notificator-idempotent-delivery`, `notificator-replay`, `replay-key-by-event-id`, `webhook-mock`, `webhook-circuit-breaker` y `circuit-breaker-client-profiles`. Otros están en curso: observabilidad, dashboards, alertas, paginación, id propio, simulador.

No todo pasa por los cuatro pasos

Cuando la decisión ya está tomada, salto pasos. En este caso fijé el breaker por cliente directamente en un apply, sin propose. La IA dejó la decisión registrada en `config.yaml` para que el próximo change la tome como restricción.



Apply directo: “fijemos ya esto: 1 breaker por client_id. No hace falta el propose.”

3. config.yaml como memoria del proyecto

La IA no recuerda nada entre sesiones. Todo lo que tiene que valer siempre vive en `openspec/config.yaml`, y OpenSpec lo inyecta en cada artefacto. Tiene tres partes:

- **Contexto:** dominio, idioma (artefactos en español, código y logs en inglés), stack cerrado, responsabilidades de cada app, permisos de DB por usuario, tópicos de Kafka, modelo de idempotencia, convenciones Java, secretos y logs.
- **Reglas por artefacto:** el proposal dice qué apps toca y qué queda fuera de alcance; el design justifica cada dependencia nueva y relaciona la seguridad con el OWASP Top 10; las specs cubren siempre el aislamiento por `client_id`; las tareas con lógica incluyen su test.
- **Guía de apply:** correr `mvn test` antes de marcar una tarea como completa y no loguear `content` ni secretos, ni siquiera en DEBUG.

Cuando una discusión termina en una decisión, la paso al config. Por ejemplo, después de comparar aspecto contra filtro para identificar al cliente quedó escrito: *"Identifica al cliente con un servlet Filter*

protegido por defecto (lista explícita de rutas públicas), nunca con aspectos ni anotaciones opt-in." Así no vuelve a aparecer en el próximo change.

4. Cómo escribo los prompts

4.1 Contrato concreto: endpoint, condiciones, códigos HTTP

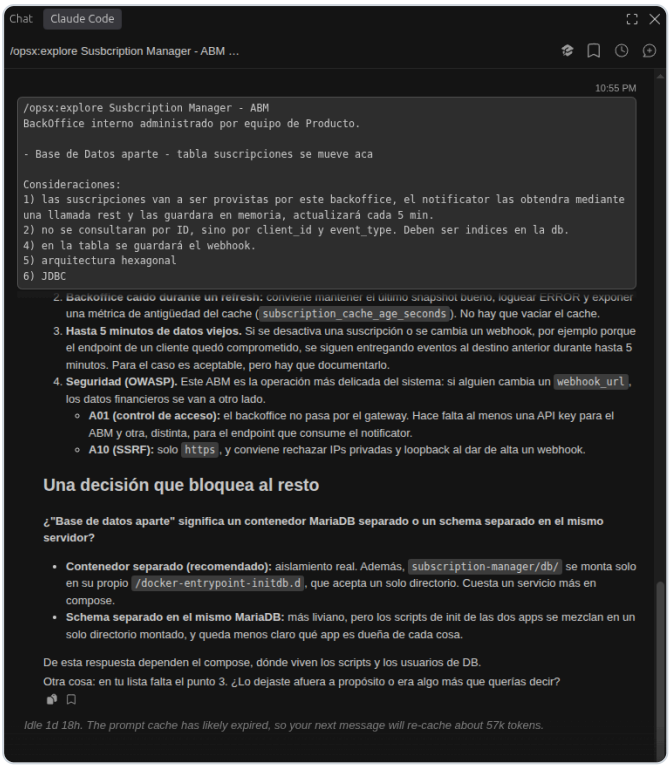
Cuanto más se parece el prompt a una spec, menos interpreta la IA. Además, como el contrato está escrito, la IA puede contrastarlo con `config.yaml`: detectó que yo pedía 400 donde el config decía 409, y dejó la decisión para que la confirmara yo.



Propose con endpoint, condiciones y códigos HTTP. Al final, dos decisiones para confirmar.

4.2 Consideraciones numeradas

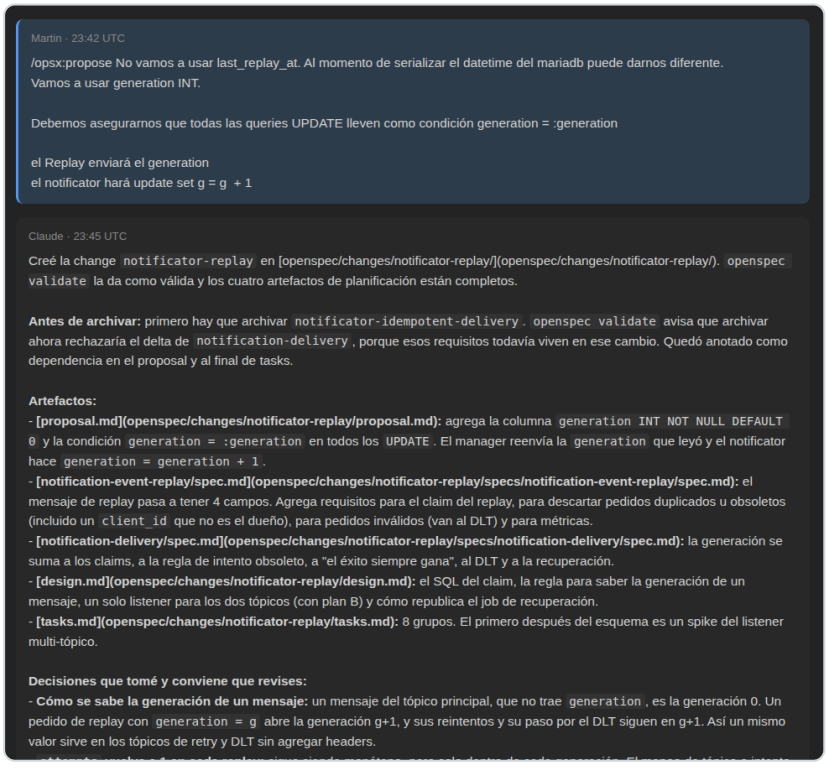
Restricciones listadas una por una. El ejemplo es el explore de un Subscription Manager, un módulo que después descarté por tiempos porque no era central para el ejercicio. La IA las respeta y es fácil ver si falta alguna: en este caso fue ella la que notó que faltaba el punto 3 y preguntó. También trajo riesgos que no había pedido (cache desactualizado, OWASP A01 y A10) y una decisión que bloqueaba el resto.



Explore del Subscription Manager (descartado) con consideraciones numeradas.

4.3 Imponer la decisión técnica cuando ya la tengo

Si sé cómo quiero resolver algo, lo digo con el porqué y con la regla que la implementación tiene que cumplir. En el explore previo la IA había recomendado `last_replay_at` como generación. Lo descarté porque la serialización del datetime de MariaDB puede no coincidir, y pedí un contador entero.



Propose que reemplaza la recomendación de la IA por un fencing token entero.

4.4 Delimitar responsabilidades y habilitar preguntas

```
/opsx:propose En manager agregar flujo replay.
Publicación:
La idempotencia del efecto es responsabilidad del notificator. El manager puede publicar
más de una vez ante pedidos concurrentes, y eso es aceptable.
Si algo es ambiguo, preguntá antes de asumir.
```

La última línea cambia el comportamiento: en vez de completar huecos con supuestos, la IA pregunta.

4.5 Pedir análisis antes de decidir

```
Respecto a la identificación del cliente. Dame un análisis sobre las implicaciones de
elegir aspectos o elegir un filter. Ventajas y desventajas de cada uno.
```

Después decidí yo: *"Vamos a usar un Filter. El filter tendrá que validar el JWT. Esto lo dejaremos para el final, por ahora que el Filter obtenga el client_id del header x-client-id."*

En explore esto pasa solo: con un prompt de una línea sobre paginación, la IA comparó offset contra cursor, recomendó uno y listó lo que dependía de esa elección. Yo respondí en dos líneas.



Paginación: la IA trae alternativas y recomendación; la decisión (“cursor”) es mía.

4.6 Partir de un artefacto

Antes de diseñar el algoritmo de entrega dibujé el flujo en Mermaid y se lo pasé como punto de partida:

```
/opsx:explore Notificator: Algoritmo
Vamos a diseñar la solución para cumplir con idempotencia contemplando escenario
concurrente con duplicidad de eventos.
Vamos a diseñar primero el flujo de las notificaciones del tópico principal.
revisa el flujo @docs/flujo-notificator.mmd
```

Ese diagrama fue evolucionando con el diseño hasta convertirse en los diagramas de flujo que están en `docs/` (entrada, primer intento, entrega, reintento, replay, recuperación y errores).

4.7 Decir qué NO hacer

```
vamos a implementar el notificator.
Mismas convenciones de código y arquitectura que el manager.
Crea una estructura base, Dockerfile y pom.xml.
No incluyas el PingController, solo el actuator.
```

4.8 Listar el resultado esperado

Para los dashboards no describí cómo hacerlos: listé qué paneles quería ver en cada uno y le pedí a la IA un relevamiento de las métricas existentes contra esa lista. Así el plan sale con los huecos de instrumentación identificados.



Explore de observabilidad: cuatro dashboards con sus paneles, como lista de entregables.

4.9 Plantear el problema, no la solución

El circuit breaker abría demasiado fácil y mi intuición era configurarlo por cliente. Planteé el problema y la necesidad, sin imponer la implementación. La IA separó dos causas que se mezclaban: la ventana por cantidad de llamadas cubre tiempos muy distintos según el volumen del cliente. Una ventana por tiempo resolvía la mayor parte, y la config por cliente quedó como complemento.



Explore del circuit breaker: el problema descompuesto antes de diseñar config por cliente.

5. Revisión crítica: no acepto la primera respuesta

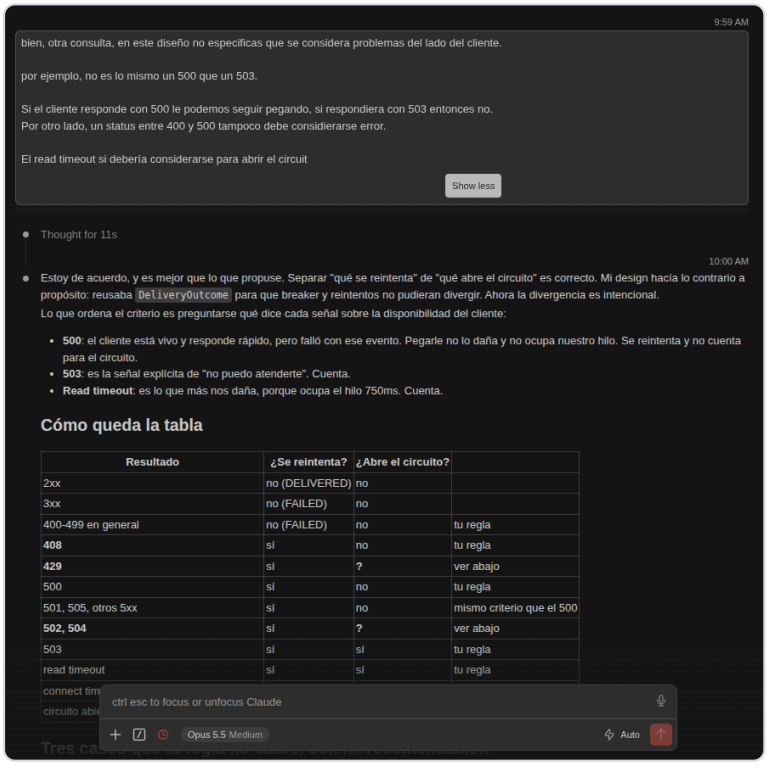
La parte de más valor fue cuestionar lo que la IA proponía. Algunos casos reales:

Lo que vi	Lo que pregunté o corregí
El circuit breaker trataba igual a todos los errores.	*"no es lo mismo un 500 que un 503. Si el cliente responde con 500 le podemos seguir pegando, si respondiera con 503 entonces no. Por otro lado, un status entre 400 y 500 tampoco debe considerarse error. El read timeout sí debería considerarse para abrir el circuit"*
Timeouts generosos para un webhook.	*"ajustemos los timeouts a como si fuera prod: llevemos el read timeout a 500ms y el connection timeout a 250ms"*
Una frase del design que no entendí.	*"a qué te referís con esto? explicámelo"*. La explicación dejó ver un error: contar una excepción propia como falla del cliente abría el circuito por un bug nuestro. Se cambió a `releasePermission()`.
Un panel de Grafana siempre en verde.	Mandé captura: *"el estado del circuit no tiene mucho sentido, siempre muestra lo mismo en verde. ¿qué representa realmente?"*. Con un scrape cada 15 s, el half-open no se ve; el panel se rehízo.

Las dos primeras filas pasaron en la misma sesión, en cuatro minutos:



Duda sobre los reintentos y ajuste de timeouts: la IA corrige el design y aplica los valores.



500 vs 503: separar “qué se reintenta” de “qué abre el circuito”. La IA lo reconoce como mejor que su propuesta.